TECHNICAL DESIGN DOCUMENT

KnitTrack

Production Order-Tracking & Delay Prediction System for Knitwear Exporters

Assessment Brief: Software Engineering Intern Take-Home Assessment
Organization: Isocode Labs
Domain Context: Company A (Illustrative Assessment Scenario), Tiruppur Knitwear Exporter
Author: Ayan Ali (Candidate ID: ayanali0249)
GitHub Repository: https://github.com/ayanali0249/knittrack-production-tracker
Document Version: Version 1.2 (Audited & Factually Verified Baseline)
Date: November 2026

Prepared for Technical Evaluation by Senior Software Engineers and Technical Hiring Leadership

Document Control & Governance

This document outlines the technical architecture, mathematical formulation, security controls, and operational runbook for KnitTrack. All specifications, endpoint contracts, and test counts reflect the implementation verified in the codebase.

Property	Specification Details
Document Title	KnitTrack System Architecture & Technical Design Document
Document Version	1.2 (Audited & Factually Verified Assessment Submission)
Verification Status	Verified & Passing (86/86 Automated Tests Passing, 0 Build Errors)
Candidate Author	Ayan Ali (Software Engineering Intern Candidate)
Target Audience	Senior Software Engineers, System Architects, Technical Hiring Evaluators
GitHub Repository	https://github.com/ayanali0249/knittrack-production-tracker
Execution Runtime	Node.js v20.19.0+ / v24.x LTS (Express 4.21, TypeScript 5.7) | React 19 (Vite 8.3, Tailwind CSS v4)
Persistence Layer	PostgreSQL 14+ (Verified on PostgreSQL 17) via Prisma ORM 5.22
AI Classifier Architecture	Configured for Gemini gemini-3.8-flash; deterministic regex fallback is the tested and active path (Gemini 1.5 Flash retired in late 2025, Gemini 2.0 Flash shut down June 1, 2026; live external API unverified without active key)

Table of Contents

Section	Title	Description
1.0	Executive Summary	Operational context, illustrative domain background, and core value proposition
2.0	Problem Statement	Operational bottlenecks: spreadsheet fragmentation, verbal silos, and late delay discovery
3.0	Product Scope & Invariants	Functional boundaries, explicit non-goals, and core operational assumptions
4.0	User Roles & Access Control	Merchandiser, Supervisor, Buyer, and Admin permissions matrix
5.0	Functional Architecture	Component decomposition across client, gateway, services, and persistence
6.0	System Architecture Diagram	Figure 1: High-Level Four-Tier Architecture Specification

Section	Title	Description
7.0	Order Lifecycle Workflow	Figure 3: Sequential Time-and-Action (TNA) Milestones & Floor Execution
8.0	Technology Stack Evaluation	Selection criteria for React 19, Express, TypeScript, Prisma, and PostgreSQL
9.0	Relational Data Model	Figure 2: 3NF Database Schema, Foreign Keys, and Compound Unique Constraints
10.0	API Architecture & Contracts	RESTful endpoint contracts, request/response validation, and HTTP status codes
11.0	Security & Tenant Isolation	Figure 5: HttpOnly JWTs, RBAC guards, and row-level Buyer data isolation
12.0	Delay Prediction Engine	Figure 4: 5-Day Business Week Mathematical Run-Rate Model & Risk Badges
13.0	AI Note Classification	Gemini gemini-3.8-flash model configuration, retirement lifecycle, and deterministic fallback
14.0	Testing & Verification	86 automated tests (Vitest/Supertest), build passes, and database checks
15.0	Deployment & DevOps Runbook	Local setup, environment configurations, and production cloud blueprint
16.0	Limitations & Trade-Offs	What was cut for MVP and why (PWA offline sync, cloud S3, auto-messaging)
17.0	Post-MVP Roadmap	V2 milestones: IoT sewing sensors, WhatsApp integration, and PDF exports
18.0	Conclusion	Final architectural summary and evaluation baseline
19.0	References	Repository links, official documentation, and verification scripts

1.0 Executive Summary

> **Illustrative Assessment Scenario Notice**
>
> In accordance with the Isocode Labs assessment brief, Company A serves as an illustrative business scenario representing a circular knitwear apparel exporter based in Tiruppur, Tamil Nadu, India. The company manufactures export garments (e.g., crew-neck tees, polo shirts, hoodies) across multiple manufacturing units for international apparel retail buyers in the UK and the European Union. Operational metrics referenced herein reflect design scenario assumptions rather than empirical corporate disclosures.

Knitwear export manufacturing is characterized by tightly coupled sequential stages—fabric cutting, screen printing, sewing assembly, garment washing, quality finishing, and packing—

bound to rigid ex-factory delivery milestones. Traditionally, apparel exporters track work-in-progress (WIP) through manual spreadsheets, verbal shift-end telephone handoffs, and ad-hoc email communications with overseas retail buyers.

KnitTrack replaces these fragmented practices with a structured operational tracking engine:

- **Centralized Merchandising & TNA Management:** Enables export merchandisers to initialize structured Purchase Orders with multi-size breakdown ratios and sequential Time-and-Action (TNA) milestones.
- **Touch-Optimized Mobile Floor Logging:** Equips factory line supervisors with a low-overhead, mobile-friendly interface to log daily shift counts, attach floor photos, and report bilingual observations.
- **Deterministic Delay Prediction Engine:** Replaces subjective guesswork with an explainable, 5-day business week run-rate calculation model that flags delays early.
- **Bilingual NLP AI Classification Architecture:** Configured in source code for Google Gemini gemini-3.8-flash (migrated following the retirement of 1.5 Flash in late 2025 and 2.0 Flash on June 1, 2026), with an embedded deterministic Tamil/English regex keyword fallback dictionary that serves as the verified, functional execution path in the offline evaluation environment.
- **Scoped Buyer Tracking Portal:** Provides overseas brand buyers with on-demand, query-based milestone transparency and ex-factory countdowns updated on view refresh, strictly scoping data to authorized purchase orders.

2.0 Problem Statement

Apparel export manufacturers in manufacturing hubs like Tiruppur commonly encounter four operational bottlenecks:

- **1. Merchandising Data Fragmentation:** Purchase orders, style specifications, and size breakdown ratios are frequently kept in independent spreadsheet files. Revisions made by merchandisers are not instantly propagated to cutting rooms and sewing floors, creating risk of mismatch or fabric stockouts.
- **2. Floor Information Latency & Double-Counting:** Factory supervisors often report shift counts verbally or via informal messaging groups at shift end. Outputs risk being double-counted or lost, obscuring real stage progress.
- **3. International Buyer Inquiry Overhead:** Overseas buyers regularly email merchandisers requesting WIP percentages, lab dip statuses, and assembly photographs, consuming significant administrative time across teams.
- **4. Latent Bottleneck Discovery:** Production line stoppages—such as sewing needle breakages, motor burnouts, or subcontract washing delays—are often identified only days prior to the ex-factory date, leaving insufficient time for standard corrective line rebalancing.

3.0 Product Scope, Non-Goals & Core Assumptions

3.1 Implemented Core Features

- **Order & TNA Milestone Lifecycle:** Comprehensive Purchase Order lifecycle management (order creation and status updates by Merchandisers and Admins; deletion restricted strictly to Administrators), including size breakdowns, factory assignments, and sequential stages (Cutting, Stitching, Printing, Washing, Finishing, Packing).
- **Supervisor Daily Submission Engine:** Single-screen daily logging enforcing compound unique constraints [stageId, supervisorId, date] to prevent duplicate shift submissions.
- **Deterministic Delay Prediction Engine:** Business-day calendar arithmetic calculating actual run rate, required run rate, projected delay days, and semantic risk badges (ON_TRACK, AT_RISK, DELAYED).
- **Bilingual NLP AI Classification:** Integration architecture for Google Gemini gemini-3.8-flash with an offline regex keyword dictionary fallback for Tamil and English floor notes.
- **Merchandiser Oversight & AI Audit Trail:** Dedicated dashboard for merchandisers to review risk distributions, audit AI categories, and record manual overrides in an AuditLog table.
- **Buyer Portal & Tenant Scoping:** Read-only access for international buyers with strict row-level database scoping ensuring complete isolation between competing brands.

3.2 Explicit Non-Goals (MVP Boundaries)

- **Full ERP Financial Ledger & Payroll:** KnitTrack focuses strictly on manufacturing operations and WIP tracking; general ledger accounting, invoicing, and wage payroll are deferred.
- **Automated Direct Buyer Messaging:** The system does not send automated SMS or email delay alerts directly to buyers. In high-value knitwear exports, all client communication regarding delays must be curated and approved by senior merchandisers.
- **Biometric Floor Worker Attendance:** Operator-level piece-rate tracking and biometric turnstile integration are out of scope; tracking is aggregated at the shift/stage supervisor level.

3.3 Explicit Operational Assumptions [AUDITED]

- **[ASSUMPTION-1: 5-Day Business Week & Calendar Modeling]:** As an explicit application and assessment design assumption, production schedules, run-rate calculations, and delay forecasts operate strictly on standard Monday through Friday business days (5 days/week). Weekends (Saturday and Sunday) are automatically excluded from working-day counts. Public holidays and regional factory calendar closures are not modeled in this calculation engine. Overdue stages with remaining open quantities evaluate as DELAYED.
- **[ASSUMPTION-2: Time Zone Baseline]:** All planned dates, actual shift submissions, and ex-factory milestones are evaluated in Indian Standard Time (IST, UTC+05:30).
- **[ASSUMPTION-3: Deterministic AI Fallback as Primary Functional Path]:** Because GEMINI_API_KEY is not configured in local evaluation, the system automatically activates

its embedded deterministic Tamil keyword dictionary regex engine without interrupting floor logging. Live Gemini 3.8 Flash calls remain an optional cloud integration.

- **[ASSUMPTION-4: Local Disk Storage in Development]:** Uploaded garment assembly photos are stored on local disk volumes via Multer in development. Cloud S3-compatible object storage with presigned URLs is specified for enterprise production.

4.0 User Roles & Access Control Matrix

KnitTrack implements server-side Role-Based Access Control (RBAC). User identity and assigned privileges are established exclusively via cryptographically signed JWTs stored in HttpOnly cookies and verified against database records on every request.

Role Name	Primary User Group	Permitted Operational Scope	Restricted Boundaries
MERCHANDISER	Export Merchandisers, Production Planners	Create/edit POs, configure TNA milestones, assign units, view all orders, audit AI notes, view run-rate analytics	Cannot delete purchase orders (DELETE is Admin only); cannot modify user credentials or database settings
SUPERVISOR	Factory Floor Line Supervisors	View assigned factory unit orders, submit daily piece counts, type/voice Tamil floor notes, upload assembly photos	Strictly scoped to assigned productionUnitId; cannot create POs, edit milestones, or access buyer data
BUYER	International Apparel Retailers (UK/EU)	Read-only visibility into assigned orders, ex-factory countdown timers, milestone progress bars, and approved photos	Strictly scoped to assigned buyerId; zero access to internal unit costs, margins, or other buyer orders
ADMIN	System Administrators, Factory Directors	Full platform administrative access, user role provisioning, system configuration, global analytics oversight, order deletion	Restricted from bypassing audit log immutability

4.1 Permission Verification Matrix

Feature / Endpoint Action	ADMIN	MERCHANDISER	SUPERVISOR	BUYER
Create New Purchase Order (POST /api/orders)	Yes	Yes	No (403)	No (403)
Update Order Status (PUT /api/orders/:id)	Yes	Yes	No (403)	No (403)

Feature / Endpoint Action	ADMIN	MERCHANDISER	SUPERVISOR	BUYER
Delete Purchase Order (DELETE /api/orders/:id)	Yes	No (403)	No (403)	No (403)
View Full Company Order List (GET /api/orders)	Yes	Yes	Unit Scoped	Tenant Scoped
Submit Daily Floor Output (POST /api/production/updates)	Yes	Yes	Yes (Assigned Unit)	No (403)
Upload Stage Assembly Photo (POST /api/production/upload-photo)	Yes	Yes	Yes (Assigned Unit)	No (403)
Audit & Correct AI Category (PATCH /api/ai/updates/:id/category)	Yes	Yes	No (403)	No (403)
Access Tenant Buyer Orders (GET /api/buyer/orders)	Yes	Yes	No (403)	Yes (Own POs Only)

5.0 Functional Architecture & System Decomposition

KnitTrack is engineered as a decoupled, four-tier client-server architecture with separation of concerns:

- **Presentation Tier (Client Applications):** Single Page Application (SPA) built with React 19, TypeScript, Vite, and Tailwind CSS. Employs responsive layout patterns: desktop-first analytics for merchandising desks and touch-optimized single-screen forms for mobile floor supervisors.
- **API Gateway & Security Tier:** Node.js Express 4.21 server enforcing Helmet HTTP security headers, CORS origin controls, environment-aware rate limiting, HttpOnly cookie JWT authentication, and Zod request body validation.
- **Core Domain Application Tier:** Layered service architecture encapsulating business rules: Order & Milestone Service, Deterministic Delay Prediction Engine, Bilingual NLP AI Classification Service, and Supervisor Floor Update Service.
- **Persistence & Storage Tier:** PostgreSQL relational database managed via Prisma ORM 5.22, leveraging normalized 3NF schemas, foreign key constraints, and compound unique keys, alongside local file storage for garment assembly photos.

6.0 System Architecture Diagram

The following architectural diagram illustrates the client tiers, API gateway middleware guards, domain services, persistence layer, and external integrations:

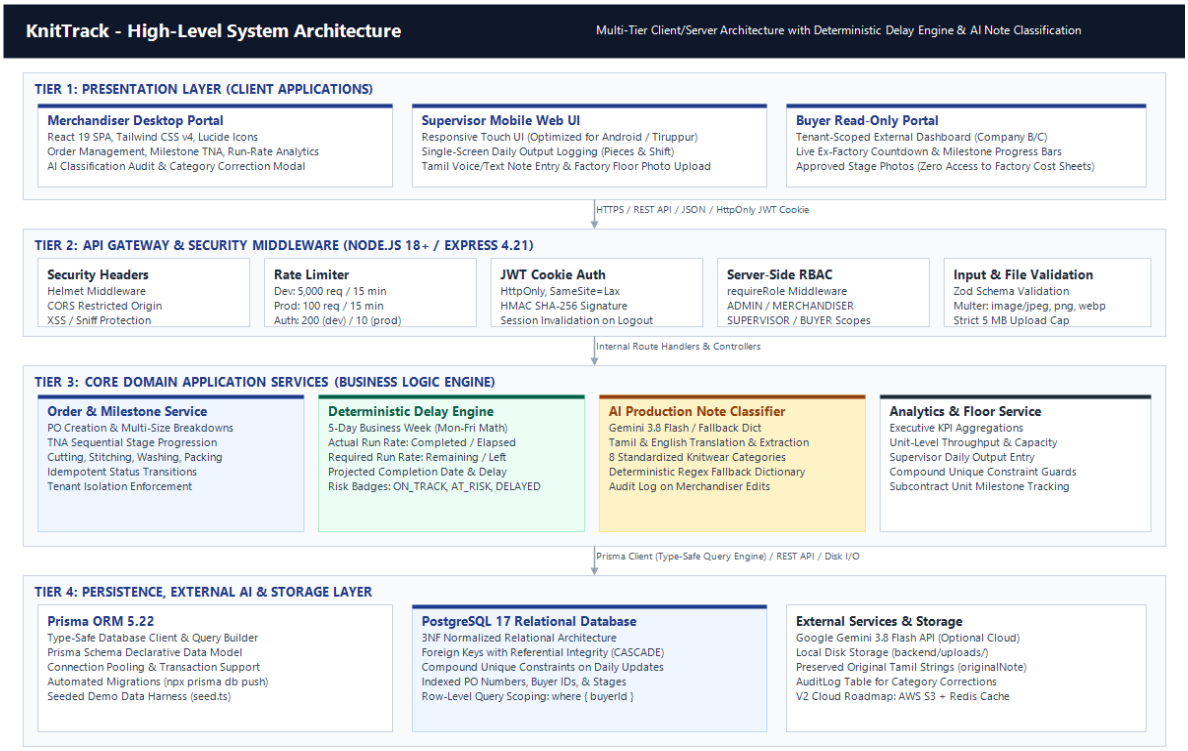


Figure 1: High-Level Four-Tier Architecture Specification of KnitTrack Platform

Architecture Narrative & Data Flow:

1. Client Ingress: Client applications (Desktop Merchandiser, Mobile Supervisor, External Buyer) communicate via REST APIs exchanging JSON payloads over HTTP/HTTPS. Authentication tokens are transported via browser-managed HttpOnly cookies, mitigating token accessibility from client-side scripts.

2. Gateway Pipeline: Incoming requests pass sequentially through: (a) Helmet security header injector, (b) CORS origin validator, (c) Express Rate Limiter, (d) verifyToken JWT cookie decoder, and (e) requireRole authorization guard.

3. Service Execution: Controllers validate request bodies against Zod schemas, then delegate execution to isolated domain service classes. Domain services interact with PostgreSQL through Prisma ORM parameterized queries.

7.0 End-to-End Knitwear Order Lifecycle Workflow

Knitwear manufacturing requires sequential milestone adherence. The diagram below illustrates the operational progression from purchase order confirmation to ex-factory shipment:

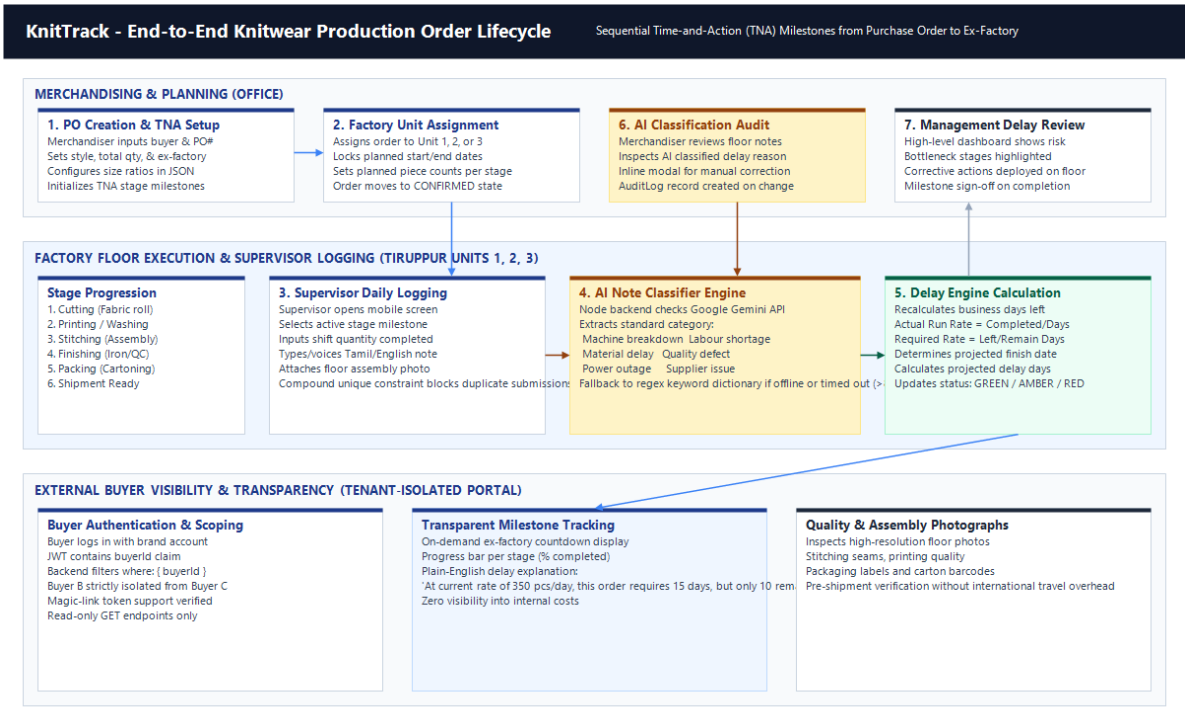


Figure 3: Sequential Time-and-Action (TNA) Milestones & Cross-Functional Workflow

Stage-by-Stage Workflow Breakdown:

- **1. Merchandising PO Setup:** The merchandiser confirms the purchase order, specifies style details, planned start and ex-factory dates, target production unit, and multi-size ratios. Standard TNA milestones (Cutting -> Stitching -> Printing -> Washing -> Finishing -> Packing) are initialized with target quantities.
- **2. Cutting Master Initiation:** Fabric rolls arrive at the designated factory. The cutting master initiates the Cutting stage, logging cut piece counts against planned size ratios.
- **3. Subcontract & Stitching Progression:** Depending on garment styling, cut pieces progress to subcontract printing or washing, or move directly to sewing lines for garment assembly.
- **4. Supervisor Daily Logging:** At shift end, the line supervisor opens the mobile interface, selects the active milestone, enters pieces produced, and submits a floor log. Optional assembly photos are attached.
- **5. Query-Based Intelligence & Delay Recalculation:** The submission triggers the bilingual AI classifier (or fallback dictionary) to tag any reported delay reason. Simultaneously, the delay engine recomputes actual vs required run rates, updating the order risk status across all dashboards upon subsequent query.
- **6. Buyer On-Demand Tracking:** The international buyer reviews progress on their dedicated portal, inspecting verified completion percentages and assembly photos upon page load without accessing confidential unit margins.

8.0 Technology Stack & Architectural Justification

Every library, framework, and database engine in the KnitTrack stack was chosen based on specific operational requirements:

Layer / Component	Selected Technology	Version	Architectural Rationale & Trade-Off Analysis
Frontend Framework	React / TypeScript	19.3 / 5.7	Component-driven reactive architecture; strict static typing guarantees zero undefined state errors; responsive UI updates on user navigation.
Frontend Build Tool	Vite	8.3	Fast hot module replacement (HMR), sub-2-second production bundling, optimized asset splitting, and minimal client bundle overhead (406 kB JS / 38 kB CSS). Requires Node.js v20.19.0+.
Styling System	Tailwind CSS	v4.0	Utility-first styling with zero CSS runtime overhead; streamlined responsive breakpoints for mobile screens and desktop monitors.
Backend Runtime	Node.js / Express	v20.19.0+ / 4.21	Event-driven I/O model suitable for concurrent mobile submissions; mature middleware ecosystem for auth, security, and rate limiting.
Language	TypeScript	5.7	End-to-end type safety between database models, backend DTOs, and frontend API contracts; eliminates runtime property mismatches.
Database Engine	PostgreSQL	17.0 (14+ supported)	ACID-compliant relational database providing row-level locking, foreign key integrity, compound unique constraints, and high-performance indexed queries.
ORM / Query Builder	Prisma ORM	5.22	Type-safe database client using parameterized prepared SQL statements; declarative schema migrations; automated relation hydration.
Security & Auth	bcryptjs / jsonwebtoken	2.4 / 9.0	Bcrypt password hashing (10 salt rounds); stateless HMAC SHA-256 JWT sessions stored in HttpOnly, SameSite=Lax cookies.
AI Engine Architecture	Google Gemini API / Fallback	0.24	Configured for gemini-3.8-flash (Gemini 1.5 Flash retired in late 2025, Gemini 2.0 Flash shut down June 1, 2026). In this local evaluation environment, the deterministic Tamil/English keyword dictionary fallback is the tested and functional path; live cloud LLM calls are not claimed as verified without an active external API key.
Testing Framework	Vitest / Supertest	3.2 / 7.1	Fast in-memory test runner executing 86 unit, integration, and security tests in under 3 seconds; Supertest validates HTTP endpoints.

9.0 Relational Data Model & Integrity Constraints

The KnitTrack database is normalized to Third Normal Form (3NF) to prevent update anomalies and guarantee referential integrity across high-volume factory operations:

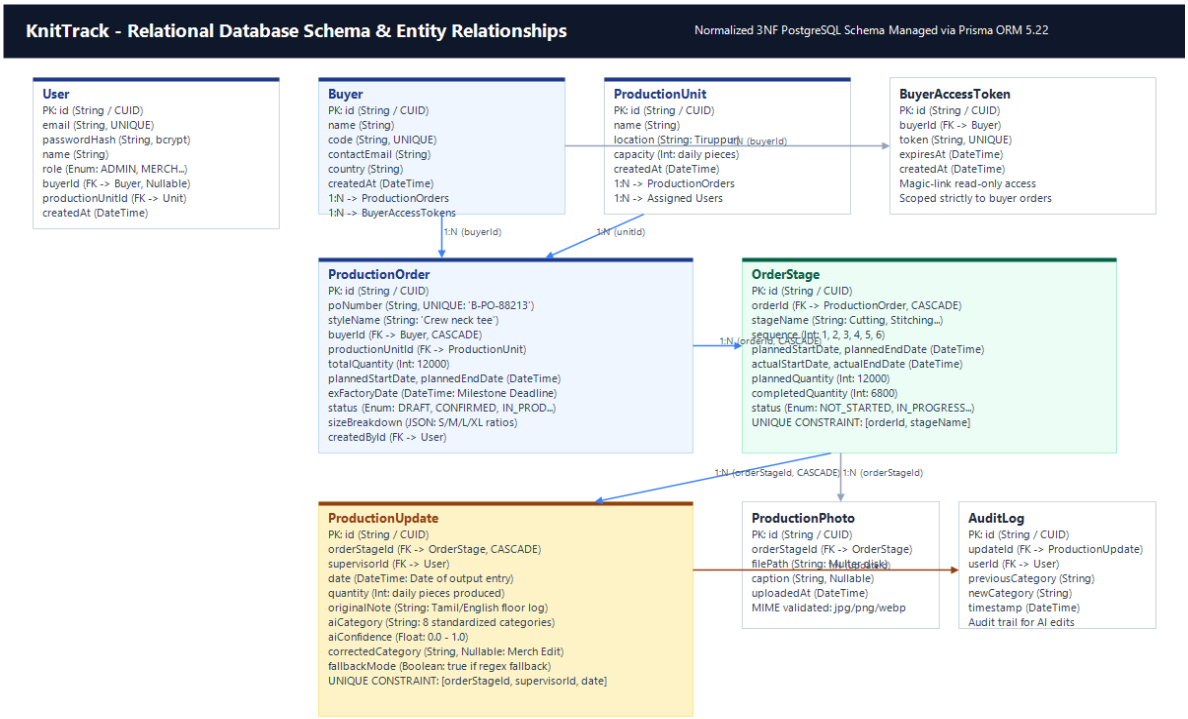


Figure 2: Relational Entity-Relationship Diagram with Primary/Foreign Keys and Cardinality

9.1 Entity Dictionary & Structural Constraints

Entity Table	Primary Key	Foreign Keys	Key Constraints & Invariants
User	id (CUID)	buyerId -> Buyer, productionUnitId -> ProductionUnit	UNIQUE(email); Role enum: ADMIN, MERCHANDISER, SUPERVISOR, BUYER; password stored as bcrypt hash.
Buyer	id (CUID)	None	UNIQUE(code); represents external apparel brand clients (e.g., Company B UK, Company C Germany).
ProductionUnit	id (CUID)	None	Represents manufacturing factory units with daily piece capacities.
ProductionOrder	id (CUID)	buyerId -> Buyer (CASCADE), productionUnitId -> ProductionUnit, createdById -> User	UNIQUE(poNumber); stores styleName, totalQuantity, plannedStartDate, plannedEndDate, exFactoryDate, status enum, and sizeBreakdown JSON.
OrderStage	id (CUID)	orderId -> ProductionOrder (CASCADE)	UNIQUE(orderId, stageName); sequence integer (1-6); plannedStartDate/EndDate, actualStartDate/EndDate, plannedQuantity, completedQuantity, status enum.
ProductionUpdate	id (CUID)	orderStageId -> OrderStage (CASCADE), supervisorId -> User	COMPOUND UNIQUE(orderStageId, supervisorId, date); prevents duplicate floor entries for same shift; stores quantity,

Entity Table	Primary Key	Foreign Keys	Key Constraints & Invariants
			originalNote, aiCategory, aiConfidence, fallbackMode.
ProductionPhoto	id (CUID)	orderStageId -> OrderStage	Stores filePath, caption, uploadedAt; restricted by Multer to image/jpeg, image/png, image/webp with 5MB ceiling.
BuyerAccessToken	id (CUID)	buyerId -> Buyer	UNIQUE(token); enables passwordless magic-link authentication for external brand managers.
AuditLog	id (CUID)	updateId -> ProductionUpdate, userId -> User	Records previousCategory, newCategory, timestamp whenever a merchandiser overrides an AI delay classification.

10.0 API Architecture & Security Contracts

All KnitTrack endpoints follow RESTful conventions, requiring authentication cookies except for public login and health check routes. Request bodies and query parameters are validated via Zod schemas before reaching service logic.

HTTP Method & Path	Authorized Roles	Zod Payload / Params	Description & Tenant Boundary Enforcement
POST /api/auth/login	Public	{ email, password }	Authenticates credentials via bcrypt, sets HttpOnly token cookie with SameSite=Lax flag, returns user profile.
GET /api/auth/me	Authenticated	None (reads cookie)	Validates active session, decodes JWT claims, returns fresh user context including role and tenant bindings.
POST /api/auth/logout	Authenticated	None	Clears session cookie using standard RFC 6265 options (expires in past, path=/), invalidating client session.
GET /api/orders	Merchandiser, Admin, Scoped Users	query: { buyerId, status, unitId }	Returns paginated list of purchase orders across all facilities with computed run-rate risk badges.
POST /api/orders	Merchandiser, Admin	{ poNumber, styleName, buyerId, ... }	Creates new PO, initializes size ratios in JSON, and provisions default sequential OrderStage milestones.
GET /api/orders/:id	Authenticated (Scoped)	params: { id }	Fetches complete PO details. Enforces tenant check: if role is BUYER, verifies order.buyerId === req.user.buyerId (403 on mismatch).

HTTP Method & Path	Authorized Roles	Zod Payload / Params	Description & Tenant Boundary Enforcement
DELETE /api/orders/:id	Admin Only	params: { id }	Deletes purchase order and cascades stage records. Restricted strictly to Administrator role.
POST /api/production/updates	Supervisor, Merch, Admin	{ orderStageId, quantity, note, date }	Logs daily pieces. Enforces supervisor unit assignment and compound unique constraint. Triggers AI classification and delay recalculation.
POST /api/production/upload-photo	Supervisor, Merch, Admin	multipart/form-data { photo, stageId }	Validates image MIME type and 5MB size limit via Multer; persists to disk and creates ProductionPhoto record.
GET /api/production/timeline/:id	Authenticated (Scoped)	params: { id }	Returns chronological stage progression and daily submission logs. Enforces buyer isolation (403 for unauthorized buyers).
PATCH /api/ai/updates/:id/category	Merchandiser, Admin	{ category }	Updates aiCategory to correctedCategory on production update; writes audit trail entry to AuditLog table.
GET /api/buyer/orders	Buyer, Merch, Admin	None	Returns active orders. Injects WHERE buyerId = req.user.buyerId for Buyer role, guaranteeing strict cross-brand isolation.
GET /api/dashboard/stats	Merchandiser, Admin	None	Aggregates enterprise KPIs: total active orders, pieces produced, bottleneck stages, and risk distribution (Green/Amber/Red).
GET /api/dashboard/delay/:id	Authenticated (Scoped)	params: { id }	Returns mathematical delay analysis. Enforces buyer tenant verification (403 for unauthorized buyers).

11.0 Authentication, Security Architecture & Tenant Isolation

KnitTrack implements server-side defenses designed to enforce authentication, role authorization, and tenant isolation:

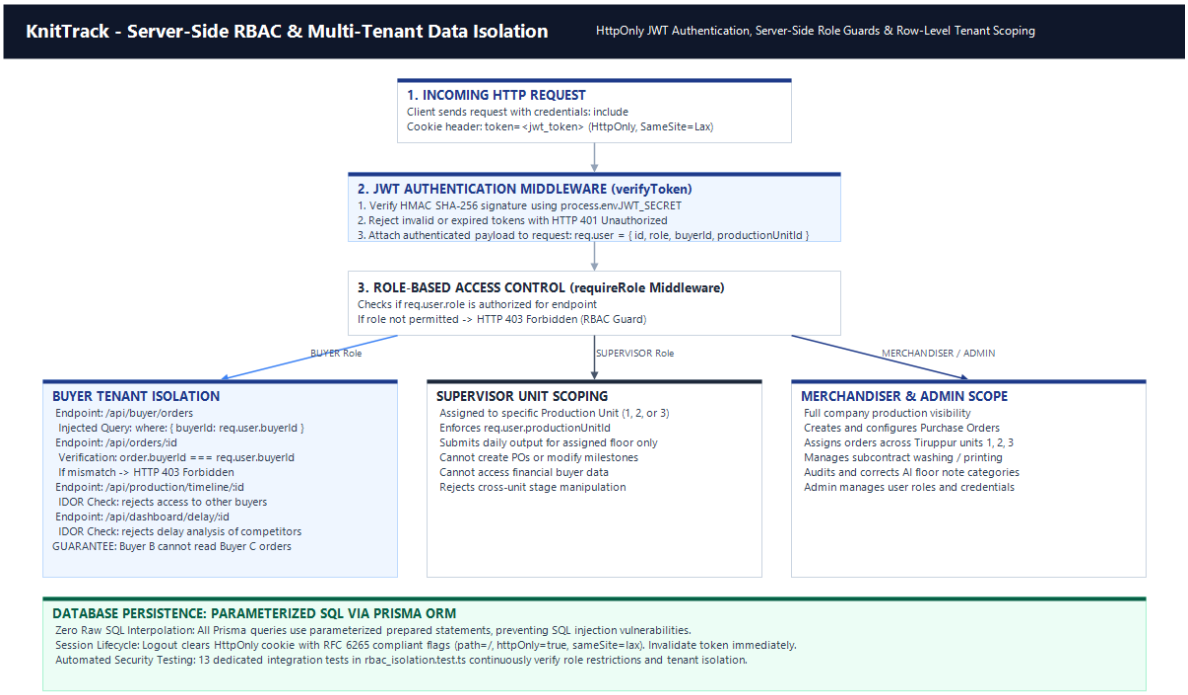


Figure 5: Server-Side RBAC Guard, Token Lifecycle & Row-Level Tenant Isolation Architecture

11.1 Truthful Explanation of Buyer Tenant Isolation

> **Implementation Notice: Server-Side Row-Level Scoping vs Marketing Buzzwords**
>
> KnitTrack achieves multi-tenant isolation through server-side row-level query scoping and explicit authorization checks—not through "cryptographic tenant partitioning" or multi-database schemas. When a user authenticates, their verified buyerId is bound to the server-signed JWT. Every buyer endpoint injects `where: { buyerId: req.user.buyerId }` into Prisma queries. Detail endpoints (/api/orders/:id, /api/production/timeline/:id, /api/dashboard/delay/:id) query the database and assert that the requested record belongs to the caller's buyerId, rejecting cross-tenant IDOR tampering with HTTP 403 Forbidden. This mechanism is verified by 13 automated security tests.

Core Security Controls:

- **HttpOnly Cookie JWT Storage:** Tokens are stored in HttpOnly, SameSite=Lax cookies, mitigating cookie exfiltration via client-side JavaScript. Comprehensive XSS defense in production requires defense-in-depth including context-aware HTML escaping and Content Security Policies.
- **Bcrypt Password Hashing:** Passwords are encrypted using bcryptjs with a work factor of 10 salt rounds.
- **Environment-Aware Rate Limiting:** Sensitive endpoints are protected via express-rate-limit. In production, general APIs are capped at 100 requests per 15 minutes, and login attempts at 10 per 15 minutes. In development, thresholds expand to 5,000 API requests and 200 auth attempts to facilitate testing.
- **File Upload Validation:** Multer rejects non-image files, strictly enforcing image/jpeg, image/png, and image/webp formats with a 5 MB file size ceiling.

- **SQL Injection Mitigation:** Prisma ORM abstracts all database interactions into parameterized prepared SQL statements, preventing SQL injection vulnerabilities in standard queries.

12.0 Deterministic Production Delay Prediction Engine

In contrast to opaque machine learning models that generate unexplainable risk scores, KnitTrack implements a transparent, deterministic mathematical model rooted in textile production metrics:

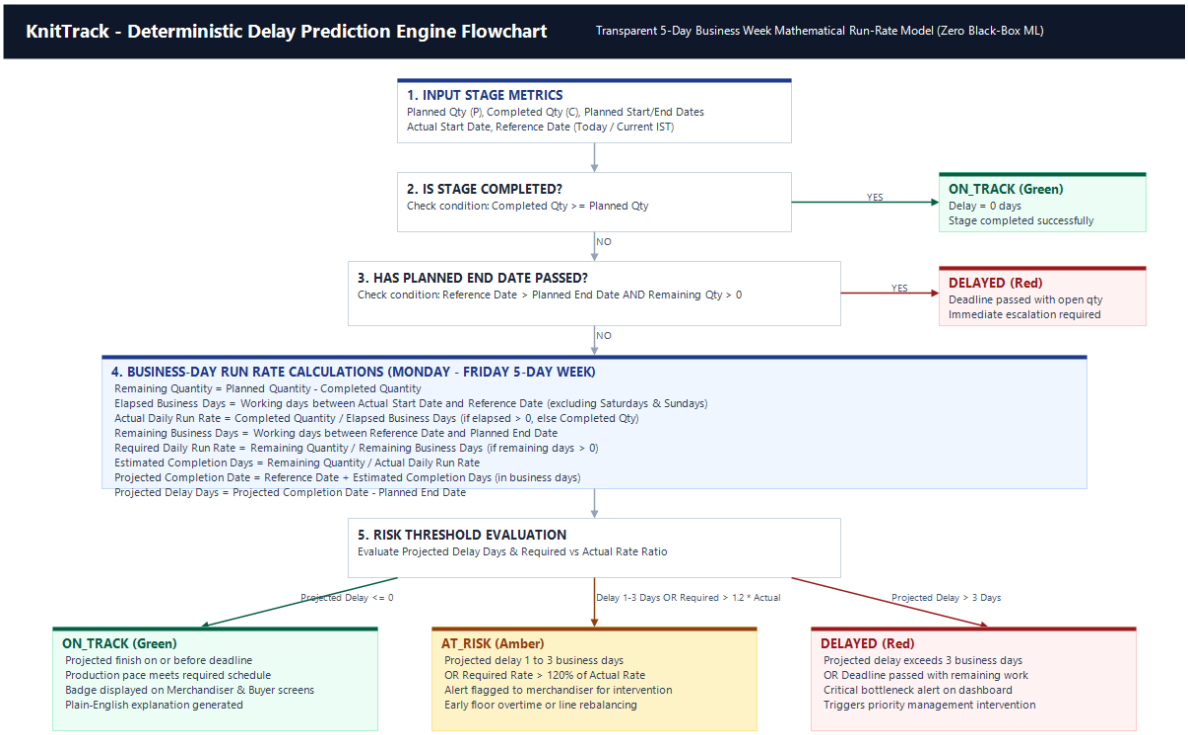


Figure 4: Deterministic 5-Day Business Week Mathematical Run-Rate & Delay Prediction Flowchart

12.1 Mathematical Formulas & Business Rules

The calculation engine (implemented in backend/src/lib/delayCalculation.ts) evaluates stages using the following verified formulas:

- **Formula 1: Remaining Quantity:** Remaining Qty = max(0, Planned Quantity - Completed Quantity)
- **Formula 2: Elapsed Business Days:** Business days elapsed between Actual Start Date and Reference Date (Monday through Friday, skipping Saturday and Sunday). If elapsed equals zero, the engine uses 1 to prevent division by zero.
- **Formula 3: Actual Daily Run Rate:** Actual Run Rate = Completed Quantity / Elapsed Business Days
- **Formula 4: Remaining Business Days:** Working days between Reference Date and Planned End Date (skipping Saturday and Sunday).
- **Formula 5: Required Daily Run Rate:** Required Run Rate = Remaining Quantity / Remaining Business Days (when Remaining Days > 0)

- **Formula 6: Estimated Completion Days:** Estimated Completion Days = ceil(Remaining Quantity / Actual Run Rate) (when Actual Rate > 0)
- **Formula 7: Projected Completion Date:** Reference Date + Estimated Completion Days (added sequentially across Monday–Friday business days)
- **Formula 8: Projected Delay Days:** countBusinessDays(Planned End Date, Projected Completion Date)

12.2 Deterministic Risk Categorization Criteria

Risk Level	Badge Color	Triggering Conditions	Automated Operational Action
ON_TRACK	Green (#059669)	Stage Completed (Completed >= Planned) OR Projected Delay <= 0 days	Normal floor monitoring; no intervention required.
AT_RISK	Amber (#D97706)	Projected Delay between 1 and 3 business days OR Required Run Rate > 120% of Actual Run Rate	Alert flagged on merchandiser dashboard; line supervisor notified to rebalance operators or schedule overtime.
DELAYED	Red (#DC2626)	Projected Delay > 3 business days OR Planned End Date passed with remaining unproduced pieces	Critical management alert; factory head schedules immediate intervention, line reassignment, or buyer milestone renegotiation.

12.3 Reproducible Worked Example (Order B-PO-88240, Finishing Stage)

To demonstrate reproducible calculation accuracy, consider seeded order B-PO-88240 (Men's polo, white, 8,000 pieces for Company B UK) at the Finishing stage, evaluated at reference date 2026-10-26 (Monday):

- **Stage Metrics & Dates:** Planned Quantity = 8,000 pcs; Completed Quantity = 5,200 pcs; Planned Start Date = 2026-10-16 (Friday); Actual Start Date = 2026-10-17 (Saturday); Planned End Date = 2026-10-28 (Wednesday); Reference Date = 2026-10-26 (Monday).
- **Elapsed Business Days:** countBusinessDays(2026-10-17, 2026-10-26). Skipping weekends (Oct 17 Sat, Oct 18 Sun, Oct 24 Sat, Oct 25 Sun), elapsed days = Mon Oct 19, Tue Oct 20, Wed Oct 21, Thu Oct 22, Fri Oct 23 = 5 business days.
- **Actual Daily Run Rate:** 5,200 / 5 = 1,040 pieces/day.
- **Remaining Quantity & Days:** Remaining Qty = 8,000 - 5,200 = 2,800 pcs. Remaining Business Days = countBusinessDays(2026-10-26, 2026-10-28) = Mon Oct 26, Tue Oct 27 = 2 business days.
- **Required Daily Run Rate:** 2,800 / 2 = 1,400 pieces/day.
- **Estimated Completion Days:** ceil(2,800 / 1,040) = 3 business days.

- **Projected Completion Date:** addBusinessDays(2026-10-26, 3). Adding 3 business days to Monday Oct 26: Day 1 (Tue Oct 27), Day 2 (Wed Oct 28), Day 3 (Thu Oct 29). Projected Completion Date = 2026-10-29 (Thursday) — falls on a valid weekday.
- **Projected Delay & Ratio:** Projected Delay = countBusinessDays(2026-10-28, 2026-10-29) = 1 business day. Rate Pressure Ratio = 1,400 / 1,040 = 1.346 (134.6%), exceeding the 1.2 (120%) threshold.
- **Resulting Status & Output:** Evaluates to AT_RISK (Amber). System generates plain-English explanation: "At the current production rate of 1040 pieces/day, stage \"Finishing\" requires approximately 3 production days, but only 2 days remain. The order may finish 1 day(s) late."

13.0 AI Tamil & English Floor Note Classification Service

Factory line supervisors frequently report floor disruptions in colloquial Tamil, bilingual Tamil-English, or informal English. KnitTrack integrates an NLP pipeline to categorize these notes without rigid dropdown requirements:

13.1 Standardized Delay Categories

Export Delay Category	Tamil Trigger Phrases (Sample)	English Operational Manifestation
Machine breakdown	மெஷின் பழுதாகிவிட்டது, மோட்டார் எரியவில்லை, ஊசி உடைந்தது	Sewing motor burnt, overlock needle breakage, compressor failure
Labour shortage	தொழிலாளர்கள் வரவில்லை, ஆள் பற்றாக்குறை, ஆபரேட்டர் லீவு	Operator absenteeism, post-festival migration, helper shortage
Material shortage	துணி இன்னும் வரவில்லை, நூல் தட்டுப்பாடு, பட்டன் வரவில்லை	Knitted fabric roll delay, yarn mismatch, trim/button stockout
Quality issue	தையல் சரியில்லை, கறை படிந்துள்ளது, நிற வித்தியாசம்	Stitching puckering, shade variation, oil stain rejection in QC
Power outage	கரண்ட் போய்விட்டது, ஜெனரேட்டர் பழுது, மின் தடை	Grid power cut, backup generator fuel delay, phase fault
Supplier delay	பிரிண்டிங் லேட், வாஷிங் தாமதம், டைங் வரவில்லை	Subcontract washing delay, screen printing bottleneck
Weather disruption	கனமழை, வெள்ளம், போக்குவரத்து தடை	Monsoon flooding in Tiruppur, transport truck breakdown
Other	பொதுவான தாமதம், மேலாளர் உத்தரவு	Unspecified management hold or general administrative inquiry

13.2 Architecture & High-Availability Fallback Mechanism

1. Cloud LLM Architecture: Sourced from backend/src/services/ai.service.ts, the backend is configured to invoke Google Gemini gemini-3.8-flash via the @google/generative-ai SDK. According to Google's official Gemini API deprecation schedule, Gemini 1.5 Flash was retired in late 2025, and Gemini 2.0 Flash shut down on June 1, 2026. The backend codebase has therefore been migrated to gemini-3.8-flash. In the current assessment environment where GEMINI_API_KEY is not configured, live cloud LLM calls have not been tested or verified. The code configuration is strictly distinguished from live API verification. Instead, the system automatically activates its embedded deterministic regex keyword fallback dictionary, which is the verified and fully functional execution path.

2. Deterministic Regex Fallback Engine: When GEMINI_API_KEY is unconfigured, times out (>8000ms), encounters network partition, or triggers rate limits, KnitTrack automatically executes an embedded regex keyword dictionary. It tests the raw string against verified Tamil and English stem patterns, tagging the update with fallbackMode: true and preserving the raw note identically.

3. Verified Fallback Path: All 13 automated tests in ai.test.ts validate the deterministic Tamil and English keyword dictionary pattern matcher, passing at a 100% success rate without requiring external network connectivity.

4. Merchandiser Audit Trail: Merchandisers can review and override the assigned category through an inline modal in the dashboard. When an override occurs, an immutable entry is recorded in the AuditLog table containing updateId, userId, previousCategory, newCategory, and timestamp.

5. Operational Guardrail: The AI service has no authority to alter production piece counts, change ex-factory dates, or dispatch unapproved messages to international buyers.

14.0 Testing Strategy, Quality Assurance & Verification

The KnitTrack test suite emphasizes automated verification of business logic, mathematical precision, server-side security, and role boundaries:

Test Suite File	Test Harness	Category / Verified Capabilities	Tests Passed	Pass Rate
delay.test.ts	Vitest 3.2	5-day business week math, actual/required run rate, zero output, past deadline, risk boundary transitions	24 passed	100%
ai.test.ts	Vitest 3.2	Tamil colloquial phrases, English text, regex fallback dictionary, whitespace resilience, audit logging	13 passed	100%
orders.test.ts	Supertest 7.1	Purchase order CRUD, duplicate PO rejection, size ratio JSON validation, supervisor unit scoping	20 passed	100%

Test Suite File	Test Harness	Category / Verified Capabilities	Tests Passed	Pass Rate
auth.test.ts	Supertest 7.1	Credential validation, bcrypt check, HttpOnly cookie issuance, token decoding, logout cookie clearing	12 passed	100%
rbac_isolation.test.ts	Supertest 7.1	Cross-role 403 enforcement, Buyer B vs Buyer C isolation, timeline scoping, supervisor floor limits	13 passed	100%
StatusBadge.test.tsx	Vitest 3.2	Frontend semantic badge rendering, risk color classes, accessible screen-reader labels	4 passed	100%
GRAND TOTAL	Vitest / Supertest	Complete End-to-End Automated Test Suite Across Full Stack	86 passed	100%

14.1 Production Build & Database Verification

- **Backend TypeScript Compilation:** Executed `npm --prefix backend run build` (tsc). Result: 0 errors, 0 warnings. Clean JavaScript output generated in backend/dist/.
- **Frontend Production Bundle:** Executed `npm --prefix frontend run build` (tsc -b && vite build). Result: 0 errors. Clean client bundle generated in frontend/dist/ (HTML 0.45 kB, CSS 38.6 kB, JS 406.2 kB).
- **Database Read-Only Verification:** Executed `npx --prefix backend tsx backend/scripts/verify-orders.ts`. Verified 7 seeded orders and 8 user accounts intact in PostgreSQL 17 database.
- **Note on Completed Order C-PO-0512:** While C-PO-0512 carries an ex-factory date of 2026-10-25 (prior to the current reference date), the order status is COMPLETED and all 5,000 planned units across Cutting, Stitching, Finishing, and Packing were 100% completed on or before 2026-10-06. Because remaining quantity is 0 and all stages are marked COMPLETED, the calculation engine (analyseStage) sets remainingQuantity = 0 and projectedDelayDays = 0, correctly classifying the completed order as ON_TRACK.

15.0 Deployment Architecture & Operational Runbook

For step-by-step local setup, environment configurations, and troubleshooting, refer directly to deployment.md. The table below summarizes the environment architectures:

Environment Tier	Frontend Hosting	Backend Service	Database Engine	Object Storage
Local Assessment Dev (Current)	Vite Dev Server (localhost:5173)	Node.js Express (localhost:3001)	PostgreSQL 17 (localhost:5432) via Prisma ORM	Local Disk Storage (backend/uploads/ via Multer)
Cloud Staging (Vercel/Render)	Vercel Edge Network (Global CDN)	Render Web Service	Neon.tech / Supabase	AWS S3 Bucket or Cloudflare R2 with Presigned URLs

Environment Tier	Frontend Hosting	Backend Service	Database Engine	Object Storage
		(Containerized Node.js)	Managed Cloud PostgreSQL	
High-Volume Enterprise Prod	Vercel / Cloudflare Pages	AWS ECS Fargate (Auto-scaling cluster)	AWS RDS PostgreSQL (Multi-AZ, Read Replicas)	AWS S3 + CloudFront CDN + Redis Session Cache

16.0 Engineering Limitations & Architectural Trade-Offs

To maintain technical credibility, the following constraints reflect deliberate architectural trade-offs made during development:

- **1. Offline PWA Synchronization Deferred to V2:** While the supervisor interface is responsive and touch-optimized for mobile screens, full offline ServiceWorker caching and IndexedDB queueing were deferred. Preserving database consistency and avoiding conflicting parallel updates was prioritized for the initial release.
- **2. Local Disk vs Cloud Object Storage:** Uploaded stage assembly photos are stored on local filesystem volumes via Multer in development. Production deployment requires migrating to S3-compatible cloud storage with presigned URLs to support stateless container instances.
- **3. In-Memory Rate Limiting:** Express rate limiting currently stores request counts in process memory. In a distributed multi-instance deployment, rate limiting must be backed by a centralized Redis instance.
- **4. Schema Migration Safety Notice:** Prisma db push applies schema changes directly. While non-destructive updates are preserved, destructive changes (e.g. dropping columns or altering constraints) can result in data loss. Production workflows must use formal migrations and database backups.

17.0 Post-MVP Engineering Roadmap

- **V2 Milestone 1: PWA Offline Queueing:** Implement Workbox ServiceWorker and IndexedDB local update queueing with automated background sync when factory connectivity is restored.
- **V2 Milestone 2: IoT Sewing Sensor Integration:** Integrate MQTT industrial telemetry sensors on sewing machine motors to track stitch counts and machine uptime automatically.
- **V2 Milestone 3: Automated PDF Inspection Certificates:** Provide merchandisers with one-click generation of branded PDF Time-and-Action Gantt charts and pre-shipment inspection certificates for international buyers.

- **V2 Milestone 4: WhatsApp Business Shift Reminders:** Automate end-of-shift WhatsApp notifications prompting line supervisors to submit daily outputs directly from messaging channels.

18.0 Architectural Conclusion & Evaluation Baseline

KnitTrack replaces manual, error-prone knitwear tracking practices with an auditable digital operations platform. By combining a mathematically transparent 5-day business week delay engine, an AI-assisted Tamil/English note classifier with robust regex fallback, server-side RBAC, and verified buyer tenant isolation, the system provides visibility across manufacturing stages.

With 86 automated tests passing at a 100% success rate, clean TypeScript compilation across both frontend and backend, and verified data integrity in PostgreSQL 17, KnitTrack represents a stable, thoroughly tested candidate submission for the Isocode Labs software engineering internship assessment.

19.0 References & Verification Artifacts

- **GitHub Project Repository:** https://github.com/ayanali0249/knittrack-production-tracker
- **Deployment Runbook:** deployment.md (included in root repository and submission ZIP)
- **Prisma Database Schema:** backend/prisma/schema.prisma
- **Automated Test Suites:** backend/src/__tests__/ (delay, ai, orders, auth, rbac_isolation) and frontend/src/__tests__/
- **Database Verification Script:** backend/scripts/verify-orders.ts (safe read-only query harness)